LAPORAN UJIAN TENGAH SEMESTER
KONSTRUKSI & EVOLUSI PERANGKAT LUNAK
LARAVEL, VUE, CI/CD GITHUB ACTIONS, DOCKER, DAN DOCKER COMPOSE



Disusun oleh:

Nama	:	Mohammad Avirza Radyatanza
NIM	:	24/534267/SV/24022
Kelas	:	PL5AA
Dosen Pengampu	:	Galih Malela Damaraji, S.Pd., M.Eng.

PROGRAM STUDI D-IV TEKNOLOGI REKAYASA PERANGKAT LUNAK

DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA

SEKOLAH VOKASI

UNIVERSITAS GADJAH MADA

YOGYAKARTA

2026

A. Informasi Umum

- Tema sistem: **Project Tracker / Todo Management** – Aplikasi manajemen proyek dan tugas untuk seorang *Project Manager* yang membawahi banyak anggota tim. PM dapat membuat proyek, menugaskan tugas ke anggota, dan memantau kemajuan.
- Repositori GitHub: https://github.com/MohammadAvirzaR/uts-pl-24-534267-SV-2
- Halaman GitHub Actions: https://github.com/MohammadAvirzaR/uts-pl-24-534267-actions
- Halaman Packages (GHCR): https://github.com/MohammadAvirzaR?tab=packages
- Postman Collection: berkas `backend/postman/ProjectTracker.postman_collection.j` pada repositori

B. Soal 1 – Backend Laravel RESTful API (CPMK1)

1) Autentikasi Token (Login dan Logout)

Autentikasi menggunakan **Laravel Sanctum**, paket bawaan Laravel 11 yang menyediakan sistem token sederhana untuk SPA dan API. Proses login dilakukan melalui endpoint `POST /api/login` yang memvalidasi email dan password, lalu membuat token menggunakan `createToken()`. Token dikembalikan dalam response JSON dan harus disertakan di header `Authorization: Bearer <token>` pada setiap request selanjutnya.

Logout dilakukan melalui `POST /api/logout` yang mencabut (*revoke*) token yang sedang digunakan melalui `currentAccessToken()->delete()`. Seluruh endpoint selain `/api/login` dan `/api/register` dilindungi oleh middleware `auth:sanctum`, sehingga mengembalikan **401 Unauthorized** apabila request tidak menyertakan token valid.

Listing 1: Potongan kode AuthController – login

```
public function login(Request $request)
{
    $request->validate([
        'email' => 'required|email',
        'password' => 'required',
```

```
    ]);

    $user = User::where('email', $request->email)->first();

    if (!$user || !Hash::check($request->password, $user->password)
        ) {
        throw ValidationException::withMessages([
            'email' => ['The provided credentials are incorrect.'],
        ]);
    }

    $token = $user->createToken('auth-token')->plainTextToken;

    return response()->json([
        'message' => 'Login successful',
        'user' => $user,
        'token' => $token,
    ]);
}
```

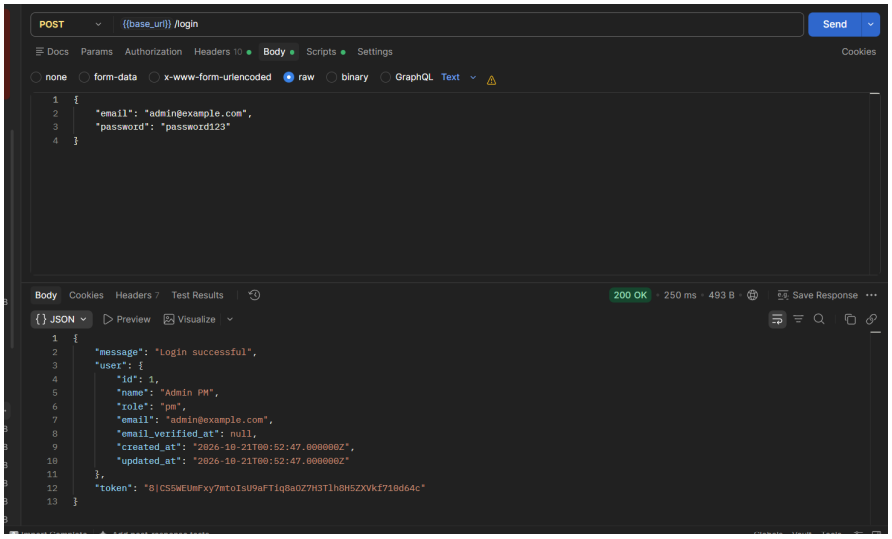


Figure 1: Postman: login berhasil dan token diterima

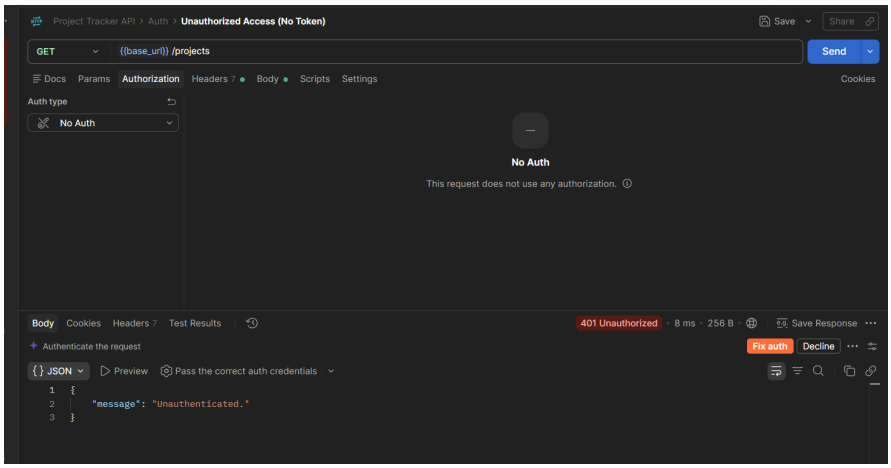


Figure 2: Postman: akses tanpa token mengembalikan 401

2) Resource CRUD

Dua resource utama disediakan: `Project` dan `Task`. Berikut tabel endpoint lengkap:

Method	Endpoint	Fungsi	Status
POST	/api/register	Registrasi user baru	201
POST	/api/login	Login, dapatkan token	200
POST	/api/logout	Logout, cabut token	200
GET	/api/user	Profil user saat ini	200
GET	/api/projects	Tampil semua project	200
POST	/api/projects	Buat project baru	201
GET	/api/projects/{id}	Detail satu project	200
PUT	/api/projects/{id}	Ubah project	200
DELETE	/api/projects/{id}	Hapus project	200
GET	/api/tasks	Tampil semua task	200
POST	/api/tasks	Buat task baru	201
GET	/api/tasks/{id}	Detail satu task	200
PUT	/api/tasks/{id}	Ubah task	200
DELETE	/api/tasks/{id}	Hapus task	200

Table 1: Daftar endpoint RESTful API

Listing 2: Potongan kode ProjectController – store

```
public function store(Request $request)
```

```
{
    $validated = $request->validate([
        'name' => 'required|string|max:255',
        'description' => 'nullable|string',
        'status' => 'sometimes|in:planning,active,completed,on_hold
            ',
        'deadline' => 'nullable|date',
    ]);

    $project = $request->user()->projects()->create($validated);

    return response()->json([
        'message' => 'Project created successfully',
        'data' => $project,
    ], 201);
}
```

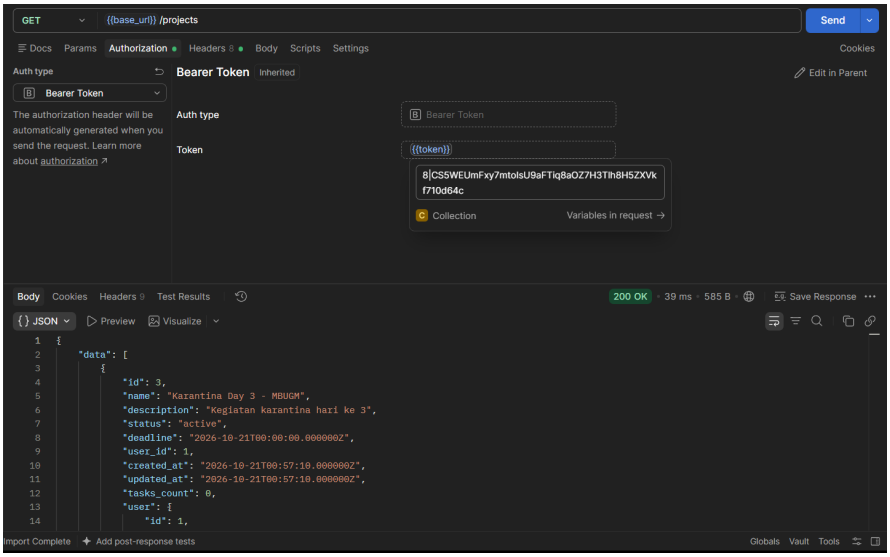


Figure 3: Postman: GET daftar data

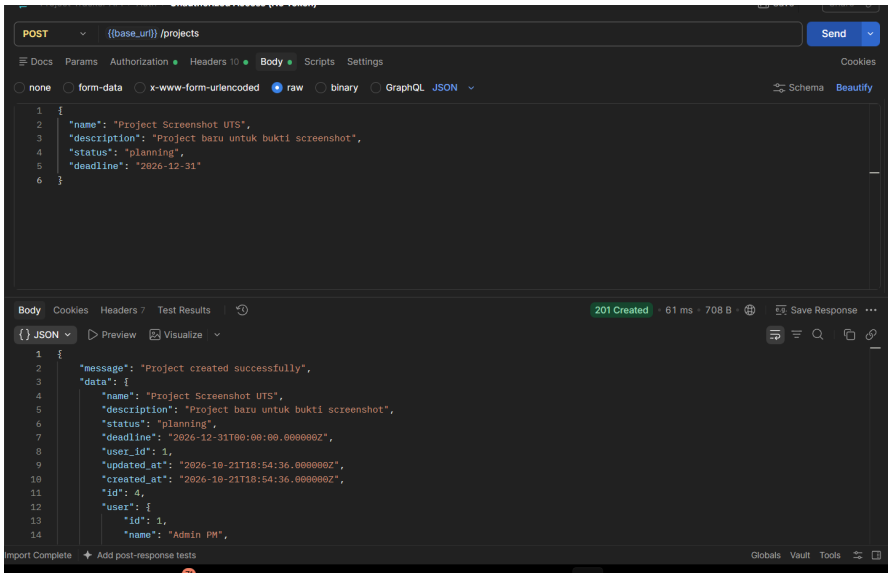


Figure 4: Postman: POST data baru, status 201

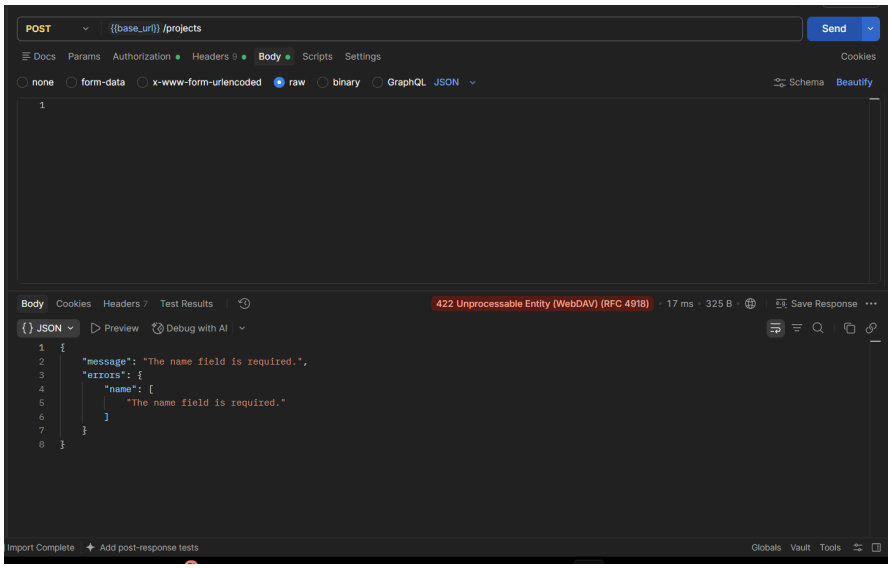


Figure 5: Postman: validasi gagal, status 422

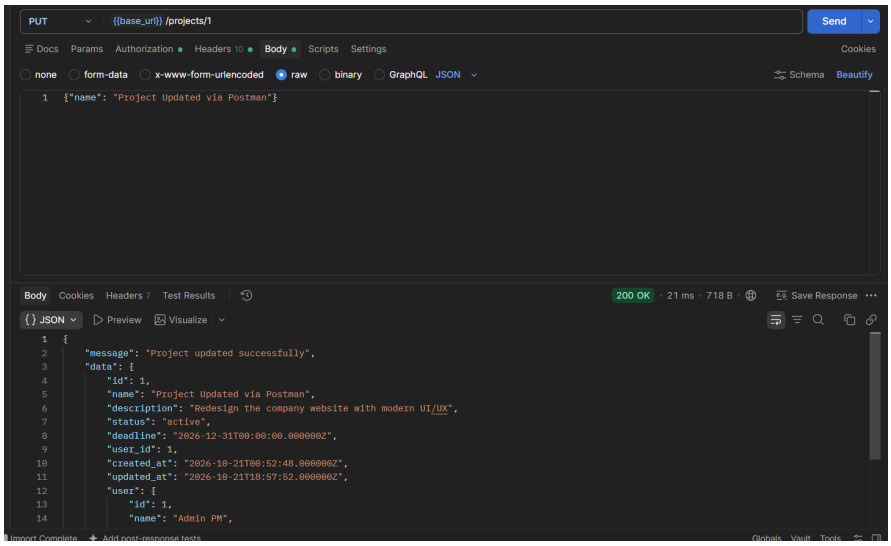


Figure 6: Postman: PUT/PATCH dan DELETE

3) Feature Test

Dua file feature test dibuat:

a. `AuthTest.php` – Menguji: login berhasil mengembalikan token, akses tanpa token mengembalikan 401, dan logout berhasil mencabut token.

b. `ProjectTest.php` – Menguji: list project (200), create project (201), validasi gagal (422), update project (200), dan delete project (200).

Listing 3: Potongan AuthTest.php

```
public function test_user_can_login(): void
{
    $user = User::factory()->create([
        'password' => Hash::make('password123'),
    ]);

    $response = $this->postJson('/api/login', [
        'email' => $user->email,
        'password' => 'password123',
    ]);
```

```
    $response->assertStatus(200)
             ->assertJsonStructure(['token', 'user']);
}

public function
   test_user_cannot_access_protected_route_without_token(): void
{
    $response = $this->getJson('/api/projects');
    $response->assertStatus(401);
}
```

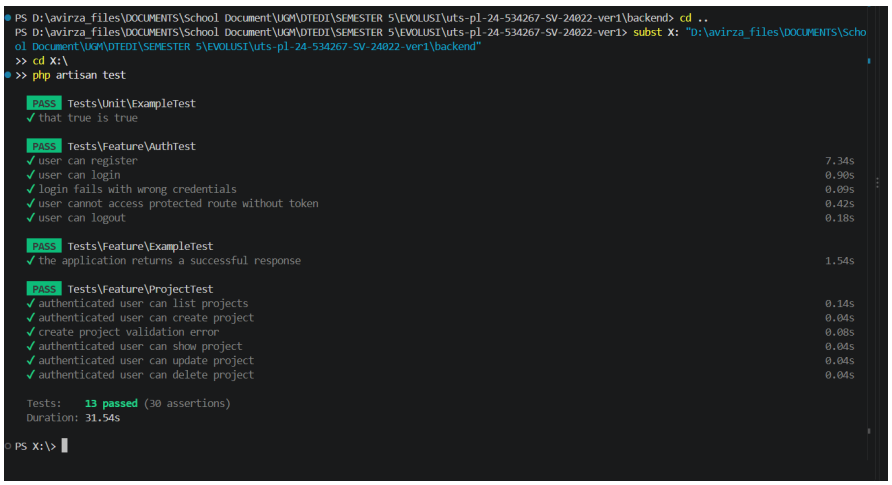


Figure 7: Output php artisan test (lulus)

4) Postman Collection

Seluruh endpoint diekspor sebagai Postman Collection dan disimpan di direktori `backend/postman/` dalam repositori.

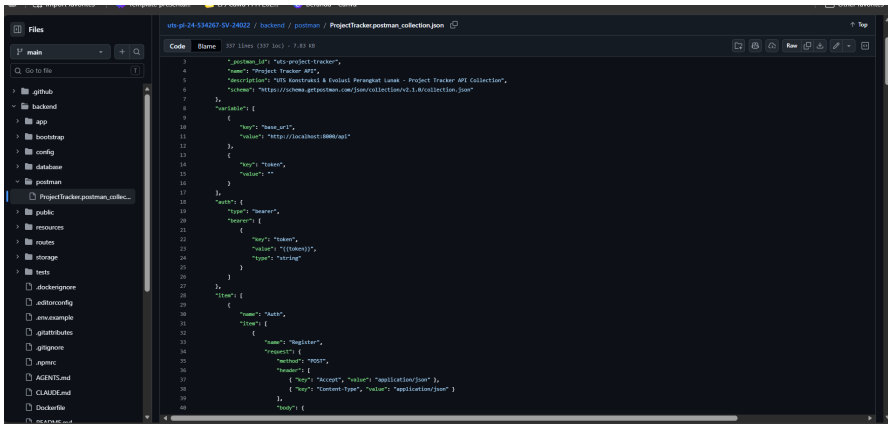


Figure 8: Berkas Postman Collection di dalam repositori

C. Soal 2 – Frontend Vue.js (CPMK1)

1) Halaman Login dan Penyimpanan Token

Token dari response login disimpan di `localStorage` melalui Pinia store (`auth.js`). Sebuah Axios interceptor dikonfigurasi untuk secara otomatis menyertakan header `Authorization: Bearer <token>` pada setiap request, sehingga komponen tidak perlu menangani token secara manual.

```
import axios from 'axios'

const api = axios.create({
    baseURL: import.meta.env.VITE_API_URL || 'http://localhost
        :8000/api',
})

api.interceptors.request.use((config) => {
    const token = localStorage.getItem('token')
    if (token) {
        config.headers.Authorization = `Bearer ${token}`
    }
    return config
})

export default api
```

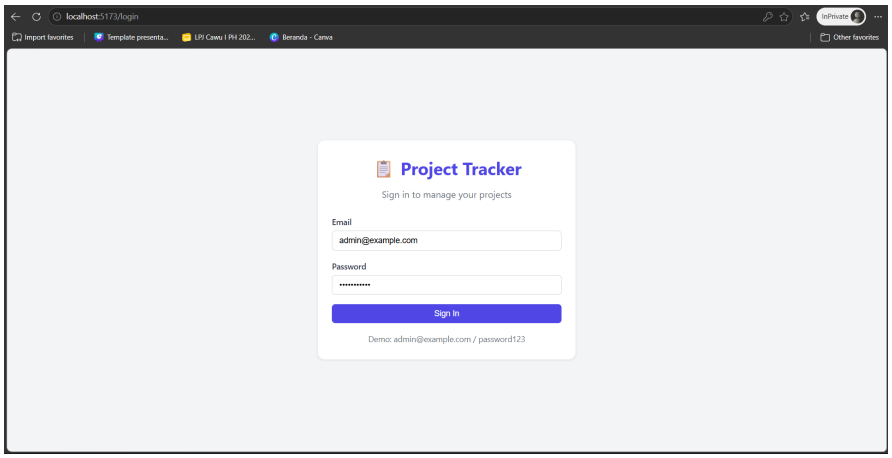


Figure 9: Halaman login Vue

2) Halaman CRUD melalui Axios

Halaman utama menampilkan daftar project milik user. Dari halaman tersebut, user dapat melihat detail, membuat baru, mengedit, dan menghapus project. Setiap project juga memiliki halaman detail yang menampilkan task-task di dalamnya.

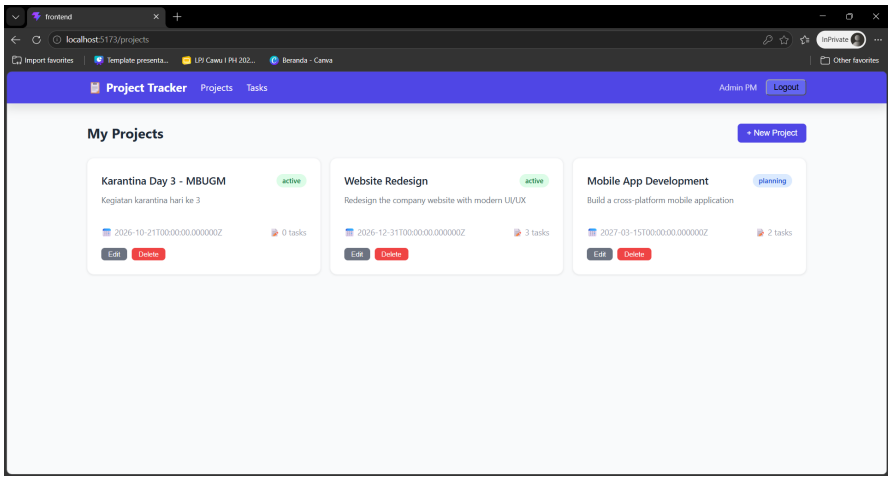


Figure 10: Halaman tampil data

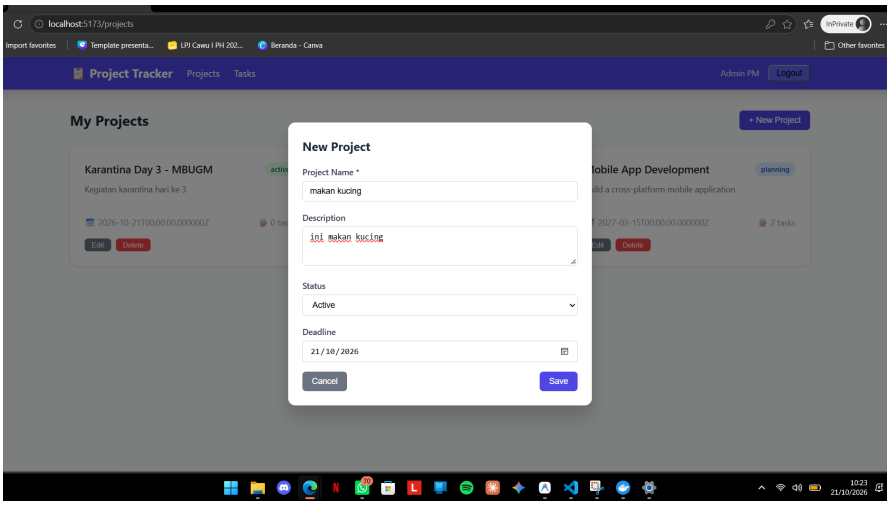


Figure 11: Form tambah dan ubah data

3) Alamat API melalui Variabel Lingkungan

Alamat API backend **tidak ditulis mati** (hardcode) di dalam kode komponen. Alamat tersebut dikonfigurasi melalui variabel `VITE_API_URL` pada berkas `.env` frontend. Variabel ini dibaca melalui `import.meta.env.VITE_API_URL` oleh Vite pada saat build. Hal ini memudahkan pergantian alamat API tanpa menyunting kode sumber, misalnya saat berpindah dari environment development ke Docker Compose.

Listing 4: File .env frontend

```
VITE_API_URL=/api
```

D. Soal 3 – Manajemen GitHub dan Continuous Integration (CPMK2)

1) Struktur Repositori dan Branch

Repository publik dibuat dengan nama `uts-pl-24-534267-SV-24022-ver1`, berisi dua folder: `backend/` untuk Laravel dan `frontend/` untuk Vue. Tiga branch digunakan: `main` (produksi), `dev` (integrasi), dan branch `feature/*` untuk pengembangan fitur.

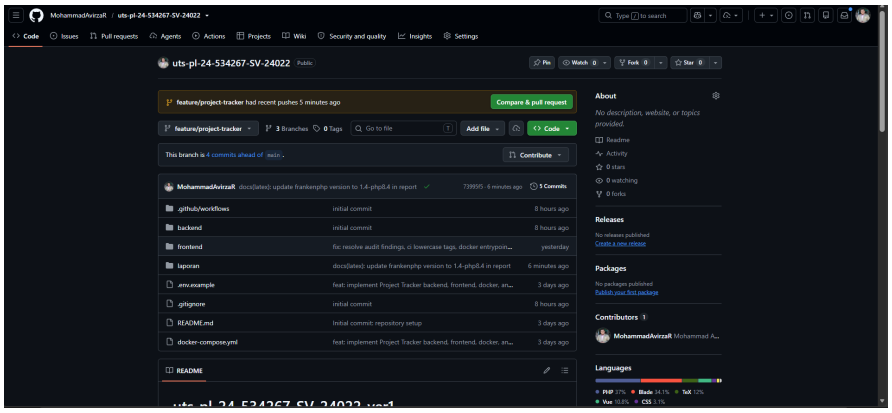


Figure 12: Tab Code repositori: folder backend/ dan frontend/

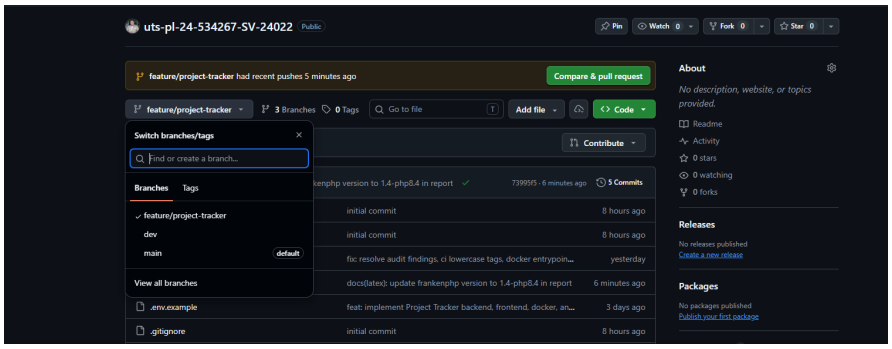


Figure 13: Daftar branch main, dev, dan branch fitur

2) Pull Request

Semua perubahan masuk ke `dev` dan `main` melalui Pull Request, tidak pernah push langsung.

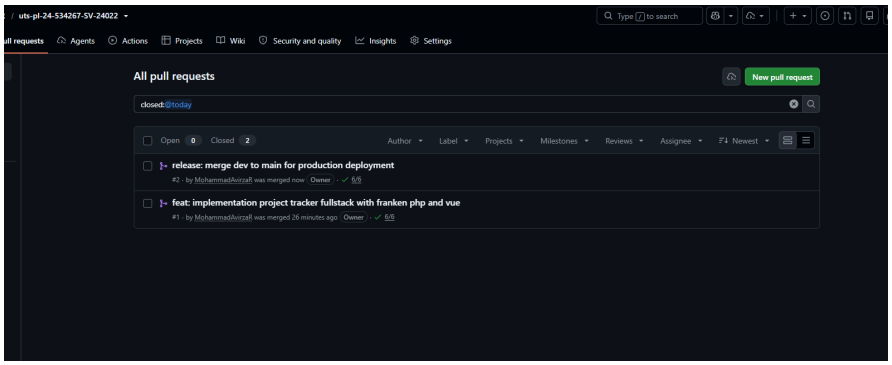


Figure 14: Pull Request yang di-merge ke dev dan main

3) Branch Protection

Branch protection diaktifkan pada `main` dan `dev` melalui Settings → Branches (atau Rules). Require Pull Request dan status check lulus sebelum merge.

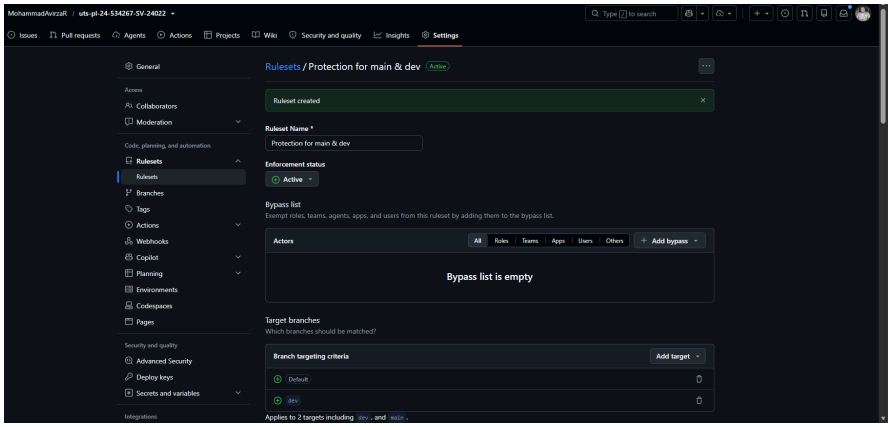


Figure 15: Pengaturan branch protection (Settings → Branches/Rules)

4) Workflow CI

Workflow `.github/workflows/ci.yml` berisi minimal dua job:

a. **test**: Menjalankan `php artisan test` untuk menguji backend.

b. **build**: Menjalankan `npm ci` dan `npm run build` untuk memvalidasi frontend.

Workflow berjalan pada setiap push dan pull request ke branch `main` dan `dev`.

```
name: CI/CD Pipeline
on:
  push:
    branches: [main, dev]
  pull_request:
    branches: [main, dev]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
```

```
      - uses: actions/checkout@v4
      - name: Setup PHP
        uses: shivammathur/setup-php@v2
        with: { php-version: '8.3' }
      - run: composer install --no-progress
        working-directory: backend
      - run: cp .env.example .env && php artisan key:generate
        working-directory: backend
      - run: php artisan test
        working-directory: backend

  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '22' }
      - run: npm ci
        working-directory: frontend
      - run: npm run build
        working-directory: frontend
```

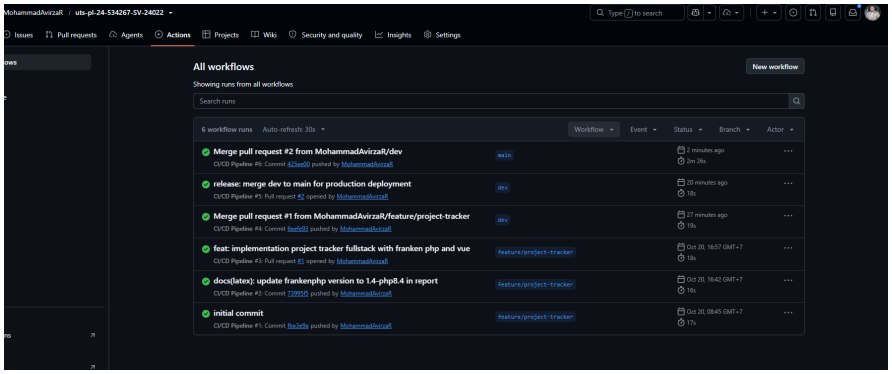


Figure 16: Workflow CI berstatus hijau pada tab Actions

E. Soal 4 – Continuous Deployment (CPMK3)

1) Kondisi Job Deploy

Job `deploy` ditambahkan pada workflow yang sama, dengan kondisi `if: github.ref == 'refs/heads/main' && github.event_name == 'push'` sehingga hanya berjalan saat ada push ke branch `main`. Job ini bergantung (`needs`) pada job `test` dan `build`, sehingga deploy hanya terjadi setelah pengujian dan build berhasil.

2) GitHub Secrets

Nilai rahasia `APP_KEY` dan `DB_PASSWORD` disimpan di GitHub Secrets (Settings → Secrets and variables → Actions). Nilai-nilai ini **tidak** ada di dalam kode repository.

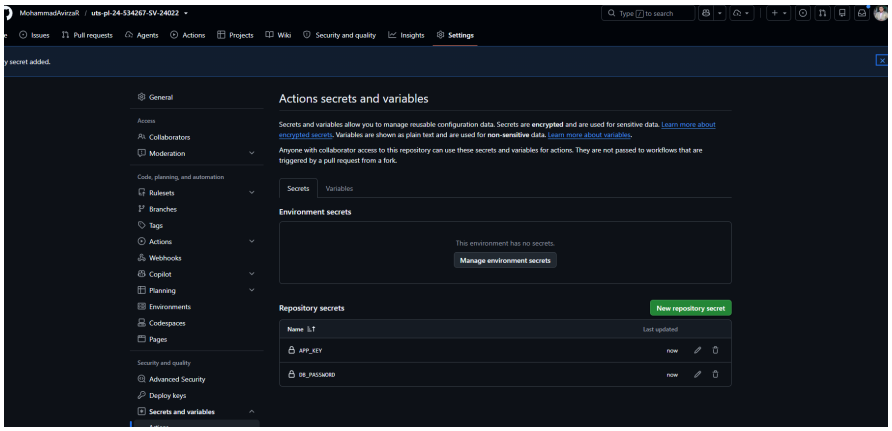


Figure 17: Daftar GitHub Secrets (nilai tidak terlihat)

3) Bukti Run

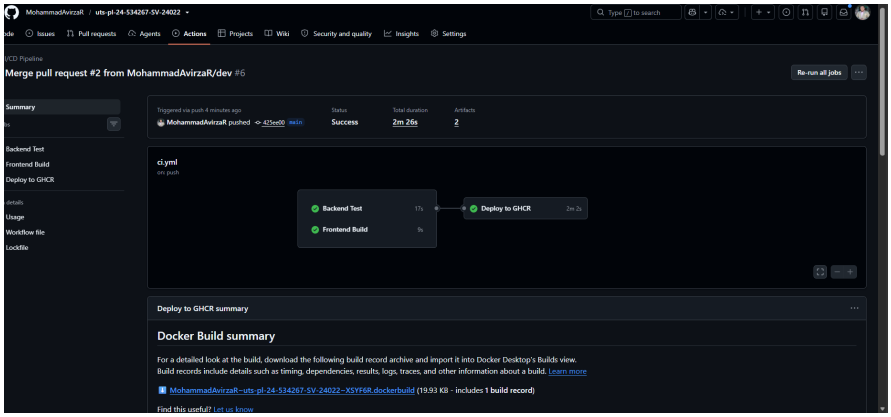


Figure 18: Run pada branch main dengan job deploy hijau

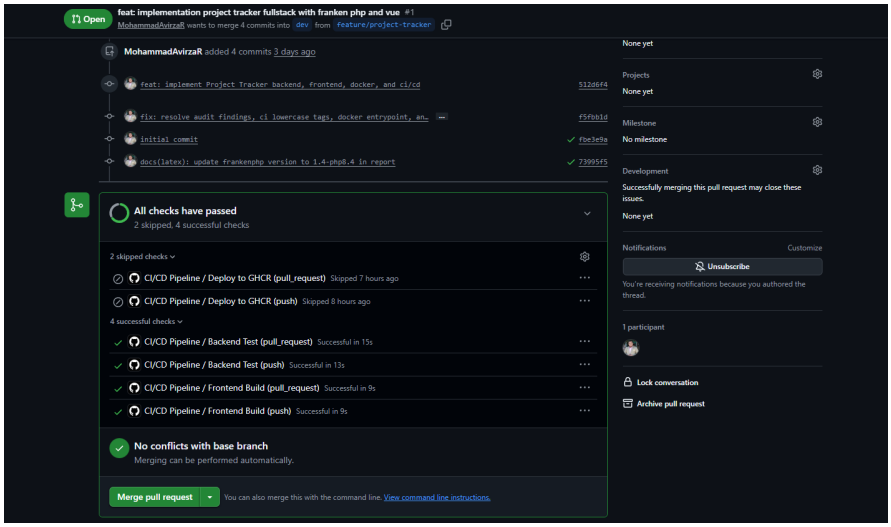


Figure 19: Run dari dev atau Pull Request: job deploy tidak berjalan

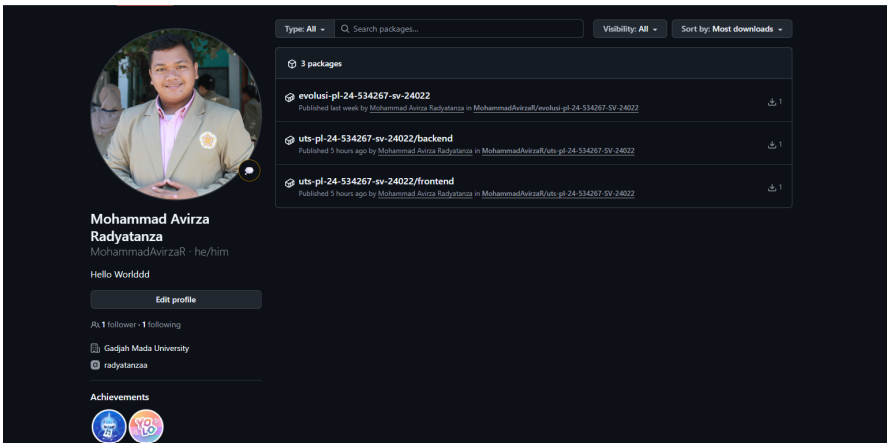


Figure 20: Halaman Packages berisi image backend dan frontend

F. Soal 5 – Dockerfile (CPMK4 dan CPMK5)

1) Dockerfile Backend

Backend menggunakan image dasar `dunglas/frankenphp:1.4-php8.4` (versi eksplisit, bukan `latest`). File `composer.json` dan `composer.lock` disalin **terlebih dahulu** sebelum kode aplikasi, lalu `composer install` dijalankan. Dengan demikian, selama dependensi tidak berubah, layer cache Composer dapat digunakan kembali dan proses build menjadi jauh lebih cepat.

Listing 5: backend/Dockerfile (FrankenPHP)

```
FROM dunglas/frankenphp:1.4-php8.4

RUN install-php-extensions pdo_pgsql pcntl

WORKDIR /app

# Layer cache: install dependencies first
COPY composer.json composer.lock ./
RUN composer install --no-dev --no-scripts --prefer-dist

# Then copy app code
COPY . .
RUN composer dump-autoload --optimize
```

```
RUN php artisan config:cache \
    && php artisan route:cache

EXPOSE 8000

CMD ["php", "artisan", "octane:frankenphp", "--host=0.0.0.0", "--
   port=8000"]
```

2) Dockerfile Frontend (Multi-Stage)

Frontend menggunakan *multi-stage build* dengan dua tahap:

1. **Tahap build** (`node:22-alpine`): menjalankan `npm ci` dan `npm run build` untuk menghasilkan file statis di `dist/`.
2. **Tahap runtime** (`nginx:1.27-alpine`): hanya menyalin hasil build dari tahap pertama. Image akhir **tidak berisi** `node_modules`.

Listing 6: frontend/Dockerfile (Multi-stage)

```
# Stage 1: Build
FROM node:22-alpine AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
RUN npm run build

# Stage 2: Serve
FROM nginx:1.27-alpine AS runtime
COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

3) .dockerignore dan Tidak Ada Rahasia

Berkas `.dockerignore` dibuat di masing-masing folder untuk mengecualikan:

- `backend/`: `vendor/`, `.env`, `storage/logs/`, `node_modules/`, `tests/`
- `frontend/`: `node_modules/`, `dist/`, `.env`, `.env.local`

Tidak ada nilai rahasia (password, key, token) yang ditulis di dalam Dockerfile. Seluruh konfigurasi sensitif diinjeksi melalui variabel lingkungan saat runtime via Docker Compose.

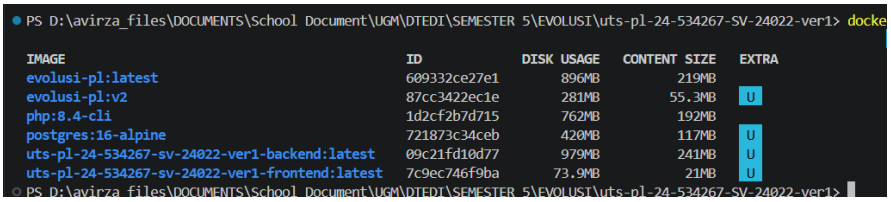


Figure 21: Output docker images: ukuran image akhir dan tanpa node_modules

G. Soal 6 – Docker Compose (CPMK6 dan CPMK7)

1) Berkas docker-compose.yml

Satu berkas `docker-compose.yml` di akar repository mendefinisikan tiga service:

a. **db**: PostgreSQL 16 dengan named volume `pgdata` untuk persistensi data.

b. **backend**: Laravel + FrankenPHP, terhubung ke `db` melalui nama service Docker.

c. **frontend**: Vue + Nginx, meneruskan request API ke `backend` melalui nama service.

Komunikasi antar service menggunakan **nama service di jaringan Docker**, bukan alamat IP laptop.

```
services:
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: ${DB_DATABASE}
      POSTGRES_USER: ${DB_USERNAME}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
```

```
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USERNAME}"]
      interval: 5s
      timeout: 5s
      retries: 5

  backend:
    build: ./backend
    environment:
      APP_KEY: ${APP_KEY}
      DB_CONNECTION: pgsql
      DB_HOST: db
      DB_PORT: 5432
      DB_DATABASE: ${DB_DATABASE}
      DB_USERNAME: ${DB_USERNAME}
      DB_PASSWORD: ${DB_PASSWORD}
    ports:
      - "8000:8000"
    depends_on:
      db:
        condition: service_healthy

  frontend:
    build: ./frontend
    ports:
      - "5173:80"
    depends_on:
      - backend

volumes:
  pgdata:
```

2) Menjalankan Sistem

```
docker compose up -d --build
```

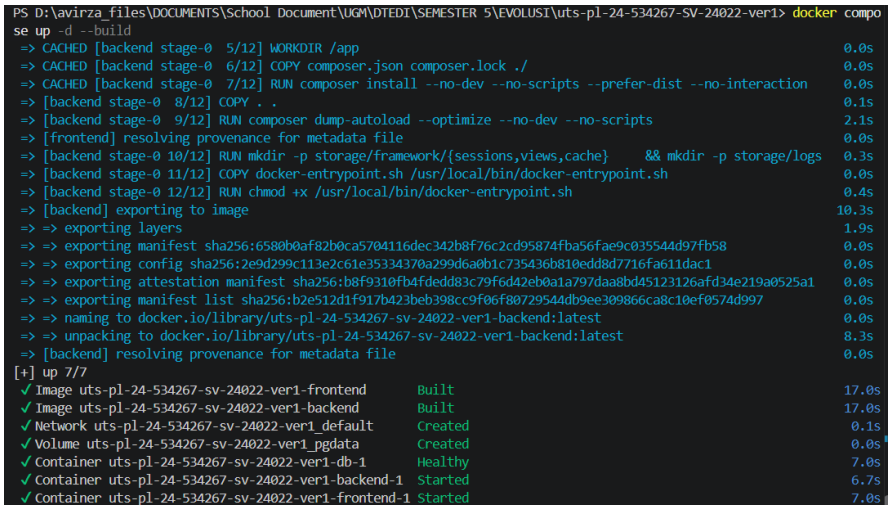


Figure 22: Output docker compose up -d –build

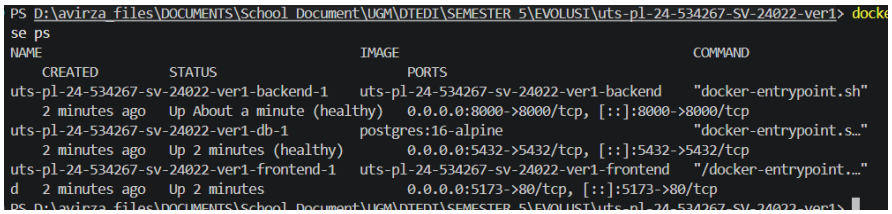


Figure 23: Output docker compose ps

3) Pengujian dari Browser

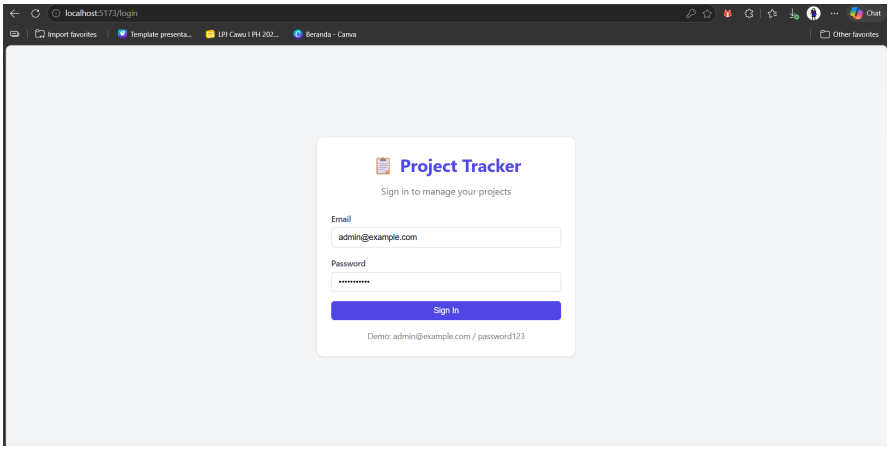


Figure 24: Login dari browser pada sistem Compose

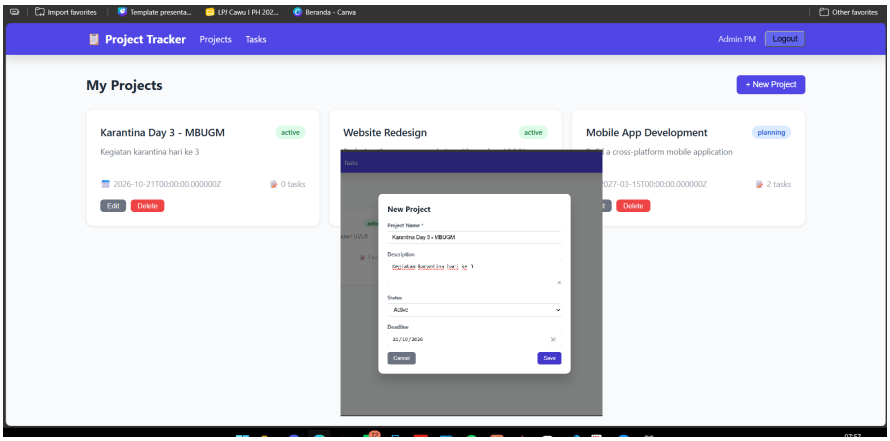


Figure 25: CRUD dari browser; data tersimpan di basis data

4) Persistensi Data

Data basis data disimpan di *named volume* `pgdata`. Setelah menjalankan `docker compose down` (tanpa `-v`) kemudian `docker compose up -d`, data yang sudah dibuat sebelumnya **tetap ada** dan tidak hilang.

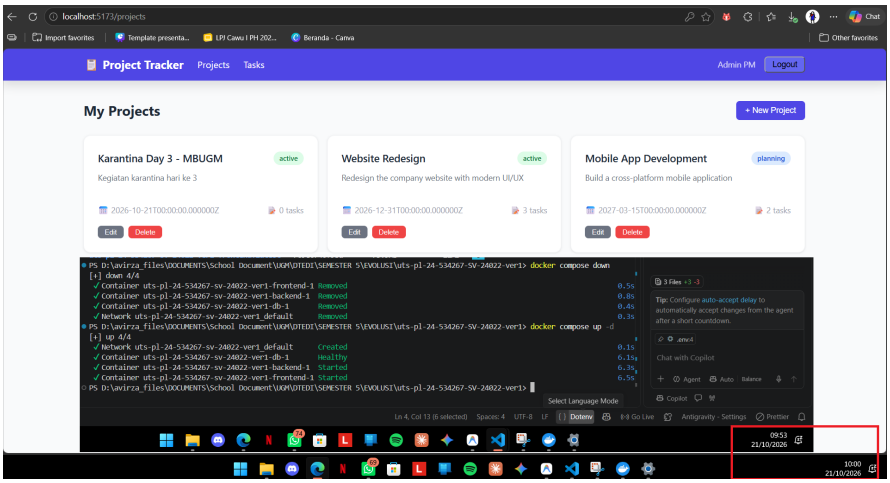


Figure 26: Data tetap ada setelah docker compose down lalu up -d

5) Kredensial dari .env

Seluruh kredensial basis data dibaca dari berkas `.env` yang **tidak** di-commit ke repository (tercantum di `.gitignore`). Berkas `.env.example` disertakan sebagai contoh konfigurasi:

Listing 7: .env.example

```
APP_KEY=base64:your-key-here
DB_DATABASE=project_tracker
DB_USERNAME=postgres
DB_PASSWORD=secret
```

H. Soal 7 – Nilai Tambahan: FrankenPHP

Backend Laravel menggunakan **FrankenPHP** sebagai server aplikasi, menggantikan `php artisan serve` atau PHP-FPM dengan Nginx. Image dasar yang digunakan adalah `dunglas/frankenphp:1.4-php8.4` (versi eksplisit, bukan `latest`).

FrankenPHP dijalankan melalui Laravel Octane dengan perintah:

```
php artisan octane:frankenphp --host=0.0.0.0 --port=8000
```

Backend tetap berjalan bersama service lain (`db` dan `frontend`) di dalam `docker-compose.yml`.

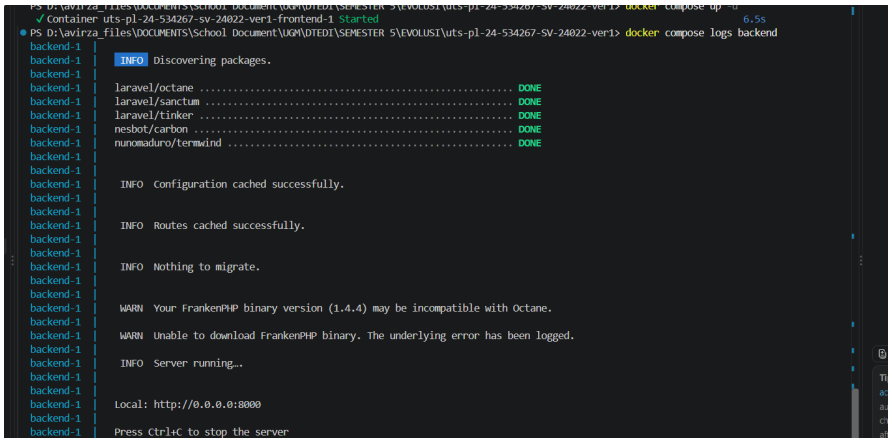


Figure 27: Log container backend menunjukkan permintaan dilayani FrankenPHP

I. Alur Data dari Vue ke Laravel ke Basis Data

1. User membuka browser dan mengakses `http://localhost:5173`. Request ini dilayani oleh container `frontend` (Nginx) yang menyajikan file statis hasil build Vue.

2. User mengisi form login. Vue mengirim `POST /api/login` ke backend melalui Axios. Request ini diteruskan oleh konfigurasi Nginx ke container `backend` melalui jaringan internal Docker Compose (menggunakan nama service `backend`, bukan IP).

3. Container `backend` (Laravel + FrankenPHP) menerima request, memvalidasi kredensial terhadap data di PostgreSQL, membuat token Sanctum, dan mengembalikannya dalam response JSON.

4. Vue menyimpan token di `localStorage`. Axios interceptor secara otomatis menyertakan token di header `Authorization: Bearer <token>` pada setiap request berikutnya.

5. Saat user melakukan operasi CRUD (misal membuat project baru), Axios mengirim `POST /api/projects` dengan data JSON ke backend.

6. Laravel memvalidasi input (mengembalikan 422 jika gagal), lalu menyimpan data ke PostgreSQL melalui Eloquent ORM. Data disimpan di named volume `pgdata` sehingga persisten.

7. Laravel mengembalikan response JSON (status 201 untuk create, 200 untuk operasi lain).

8. Vue menerima response dan memperbarui tampilan secara reaktif.

J. Kendala dan Cara Mengatasinya

Kendala	**Solusi**
CORS error saat frontend Vue mengakses backend Laravel dari container berbeda	Menambahkan konfigurasi CORS di `config/cors.php` Laravel dengan `allowed_origins` yang mencakup alamat frontend, serta memastikan `supports_credentials` diatur `true`
PostgreSQL container belum siap saat Laravel mencoba migrasi	Menambahkan `depends_on` dengan `condition: service_healthy` dan healthcheck pada service `db` di docker-compose.yml, sehingga backend menunggu database benar-benar siap
Path proyek terlalu panjang di Windows sehingga file di `vendor/` terpotong saat `composer install` (beberapa file kehilangan ekstensi `.php`)	Menggunakan `subst X: <path>` untuk memetakan path panjang ke huruf drive pendek, kemudian menjalankan ulang `composer install` dari drive `X:\`

K. Kesimpulan

Berdasarkan pengerjaan ujian tengah semester ini, dapat disimpulkan bahwa:

1. Aplikasi *Project Tracker* berhasil dibangun dengan arsitektur backend-frontend terpisah menggunakan Laravel 11 (RESTful API + Sanctum) dan Vue 3 (SPA + Axios + Pinia), saling berkomunikasi melalui API JSON dengan autentikasi token.

2. Seluruh kode dikelola menggunakan Git dengan strategi branching (`main`, `dev`, `feature/*`), branch protection pada `main` dan `dev`, dan Pull Request wajib sebelum merge.

3. Pipeline CI/CD diterapkan melalui GitHub Actions: job `test` (backend) dan `build` (frontend) berjalan pada setiap push/PR, sedangkan job `deploy` (build dan push image Docker ke GHCR dengan tag commit SHA) hanya berjalan pada push ke `main`.

4. Dockerfile backend memanfaatkan FrankenPHP sebagai server aplikasi dengan layer caching Composer, sedangkan Dockerfile frontend menggunakan multi-stage build (Node.js + Nginx) sehingga image akhir tidak mengandung `node_modules`.

5. Docker Compose berhasil mengorkestrasi tiga service (PostgreSQL, backend, frontend) dalam satu perintah `docker compose up -d -build`, dengan named volume `pgdata` untuk persistensi data dan komunikasi antar-service melalui jaringan Docker.

6. Penggunaan FrankenPHP sebagai server aplikasi Laravel (bonus) terbukti berfungsi, dibuktikan dari log container yang menunjukkan request dilayani oleh FrankenPHP.